

L11c: Markov Decision Processes

Value iteration and random rollout

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How should an agent choose its actions when each action changes both the reward it collects and the state it moves to?

Today's concepts

- Formulate a Markov decision process: define its states, actions, rewards, transition model, and discount factor, and the policy that chooses an action in each state.
- Solve for the optimal policy with value iteration: update the value of each state from the states it can reach next until the values stop changing.
- Estimate values with random rollouts: average the discounted rewards of simulated random trajectories, without writing down the transition probabilities.

Lecture: Markov Decision Processes (MDPs)

Lecture and Example

Lecture: Markov Decision Processes (MDPs)
In this lecture, we formulate Markov decision processes, solve them with value iteration, and estimate values with random rollouts.

Example: Random Rollout in the Lava Grid World
In this example, we estimate the value of every square in a lava grid world by averaging random rollouts. We then read a policy from the estimates, draw it, and follow it from a starting square to the charging station.

From Markov Chains to Decisions

A Markov decision process (MDP) extends the Markov chain by adding actions and rewards.

- A Markov chain evolves passively, according to fixed transition probabilities.
- In an MDP, an agent chooses actions that influence the state transitions and generate rewards.

MDPs are the foundation for sequential decision-making under uncertainty: the agent must make good decisions, one after another, in a stochastic environment.

MDP Components

An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, R(s,a), T(s' \mid s,a), \gamma)$:

- State space $\mathcal{S}$: all possible states s of the system.
- Action space $\mathcal{A}$: all possible actions a; the actions available in state s are $\mathcal{A}_s \subseteq \mathcal{A}$.
- Reward function $R(s,a)$: the immediate reward for taking action a in state s; r is the reward received on a single step.
- Transition model $T(s' \mid s,a) = P(s_{t+1} = s' \mid s_t = s, a_t = a)$: the probability that action a in state s leads to state s'.
- Discount factor $\gamma \in [0,1)$: weights future rewards against immediate ones; $\gamma = 0$ cares only about now, and $\gamma \to 1$ values long-term returns.

The Agent's Goal: A Policy

The agent's goal is a policy $\pi : \mathcal{S} \to \mathcal{A}$, with $\pi(s) = a$, that maps states to actions while maximizing the expected cumulative discounted reward:

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right].$$

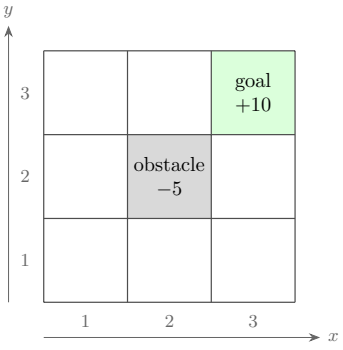
Previously, π denoted a state distribution; here it is a policy.

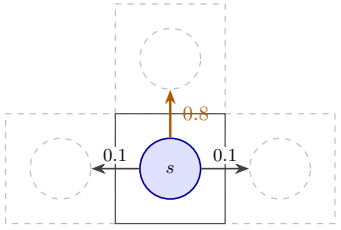
Example: Grid World Navigation

A robot navigates a 3×3 grid to reach a goal while avoiding an obstacle and minimizing travel time:

- States: the eight squares (x, y), $x, y \in \{1, 2, 3\}$, other than the obstacle at $(2, 2)$; x increases to the right and y upward.
- Actions: up, down, left, right; moves off the grid are unavailable, so $\mathcal{A}_s \subsetneq \mathcal{A}$ on the edges.
- Rewards: $+10$ for choosing a move into the goal at $(3, 3)$, -5 for choosing a move into the obstacle, and -1 for every other action.
- Transitions: slippery terrain; the intended move succeeds with probability 0.8, and each perpendicular move happens with probability 0.1.
- Discount factor $\gamma = 0.9$: the $+10$ goal outweighs the -1 step costs, so the robot pursues the goal.

Grid World: The Layout





action a = up; a move off the grid or into the obstacle leaves the robot in place

The reward depends on the action the robot chooses; slips change only where it lands. A bump or a slip off the grid leaves it in place, and the goal is terminal, so it earns zero further reward after entry.

Grid Worlds in Chemical Engineering

Divide a process's operating window into a grid, with two operating variables, such as temperature and feed flow rate, on the axes:

- States: operating points; we want to move the process to a point that meets a new product specification.
- Actions: raise or lower one operating variable by one step.
- Transitions: each action moves to a neighboring point; disturbances play the role of slips.
- Rewards: a large reward for meeting the specification, large negative rewards for violating a constraint, and a small cost per step of off-spec product.

The optimal policy says which variable to change at each operating point.

Value Iteration

Value iteration is a dynamic programming algorithm: it builds the value of each state from the values of the states that can follow it. It repeats the Bellman backup:

$$U_{k+1}(s) = \max_{a \in \mathcal{A}_s} \left(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s, a)\, U_k(s') \right)$$

- For $0 \leq \gamma < 1$, $U_k(s) \to U^*(s)$ as $k \to \infty$.
- The optimal value function $U^*(s)$ is the maximum expected cumulative discounted reward achievable from state s under the best possible policy.

Value Iteration Algorithm

Initialize: Given an MDP, a tolerance $\epsilon > 0$, and a maximum number of iterations $k_{\max}$, set $k \leftarrow 0$, $U_0(s) \leftarrow 0$ for all $s \in \mathcal{S}$, and `converged` $\leftarrow$ `false`.

While not `converged` **do**:

1. For each state $s \in \mathcal{S}$, compute the Bellman backup $U_{k+1}(s)$.
2. Check whether to stop:
   - If $\max_{s \in \mathcal{S}} |U_{k+1}(s) - U_k(s)| \leq \epsilon$ **then** set `converged` $\leftarrow$ `true` and return $U^* \leftarrow U_{k+1}$.
   - Otherwise, if $k + 1 \geq k_{\max}$ **then** return U_{k+1} with `converged` $=$ `false` and a warning.
3. Increment the iteration counter, $k \leftarrow k + 1$.

Extract Policy: for each state $s \in \mathcal{S}$,

$$\pi^*(s) \leftarrow \arg\max_{a \in \mathcal{A}_s} \left(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s' \mid s, a)\, U^*(s') \right).$$

Value Iteration: Cost

Each iteration costs $\mathcal{O}\left(|\mathcal{S}|^2\,|\mathcal{A}|\right)$ operations:

- for each of the $|\mathcal{S}|$ states, we evaluate $|\mathcal{A}|$ actions,
- and each action sums over $|\mathcal{S}|$ possible next states.

The number of iterations depends on γ and ϵ: each iteration shrinks the largest error in the value function by at least a factor of γ.

Value iteration is tractable for thousands of states, but challenging for continuous or very large discrete state spaces. There, we need function approximation or sampling-based methods.

Random Rollout

Random rollout estimates $U(s)$ by simulation:

- Start at s, choose actions at random, and add up the discounted rewards.
- Average over many rollouts to get $\hat{U}(s)$.

Nothing in that loop reads $T(s' \mid s, a)$ or $R(s, a)$. The environment only has to report a next state and a reward, so rollout runs where value iteration cannot.

Random Rollout Algorithm

Initialize: Given a simulated environment, $\gamma \in [0, 1)$, a maximum depth $d \geq 1$, and a number of rollouts $N \geq 1$, set $\hat{U}(s) \leftarrow 0$ and $n(s) \leftarrow 0$ for all $s \in \mathcal{S}$.

For $i = 1$ to N **do**:

1. Choose a starting state s_0; set $s \leftarrow s_0$, $t \leftarrow 0$, $G \leftarrow 0$, and `terminated` $\leftarrow$ `true` if s_0 is terminal, otherwise `false`.
2. While not `terminated` **do**:
   - Select a uniformly at random from $\mathcal{A}_s$; execute it and observe r and s'.
   - Update $G \leftarrow G + \gamma^t r$ and $t \leftarrow t + 1$.
   - If $t \geq d$ or s' is terminal **then** set `terminated` $\leftarrow$ `true`; otherwise, $s \leftarrow s'$.
3. Update the estimate with an incremental mean:

$$n(s_0) \leftarrow n(s_0) + 1, \qquad \hat{U}(s_0) \leftarrow \hat{U}(s_0) + \frac{1}{n(s_0)} \left(G - \hat{U}(s_0) \right).$$

Output: $\hat{U}(s)$ for every state that served as a starting state.

Random Rollout: Cost and Accuracy

Random rollout costs $\mathcal{O}(N \cdot d)$ simulated steps.

- Apart from one estimate per state, that cost does not grow with $|\mathcal{S}|$, which makes rollout usable on large or continuous state spaces.
- Accuracy comes slowly: the standard error of a sample mean falls as $\mathcal{O}(1/\sqrt{n(s)})$, so ten times the rollouts buys about three times the precision.

Example: Random Rollout in the Lava Grid World

In this example, a robot must reach its charging station without falling into a lava pit:

1. Build the lava grid world as an MDP, and shape the rewards so moves toward the station pay a little more.
2. Estimate each square's value by averaging random rollouts.
3. Read a policy from the estimates with $Q(s, a)$, and follow it from a start square.

The policy improves on random moves, but it is not the optimal policy that value iteration computes.

Example: Random rollout in the lava grid world

Lab: Value Iteration in a Grid World

In the lab, a robot must reach a charging station while avoiding lava pits:

- we build the MDP, its reward matrix and transition array, for the robot's grid world,
- we write our own value iteration with a convergence check and compare it with the course package's version, which runs a fixed number of sweeps,
- we extract the optimal policy, follow the robot's path, and see how the discount factor changes its behavior.

Lab: Value iteration in a grid world

Summary

We added actions and rewards to Markov chains, which lets an agent make optimal sequential decisions.

Key takeaways

- **MDPs.** Actions shape where the process goes, rewards score each choice, and a policy assigns an action to every state.
- **Value iteration.** Repeated Bellman backups give the optimal values; the action that achieves each maximum is the optimal policy.
- **Random rollout.** Averaging simulated random trajectories estimates how well random actions do, without the transition probabilities.

Value iteration needs the transition model; random rollout needs only a way to simulate the environment.